



DataScience_SQ01_데이터베이스 개론 및 SQL 기초

📄 BDAI 공식홈	https://bdaprogram.oopy.io/
■ Status	SQL
💬 문의	official.bdaa@gmail.com
■ 선택	



데이터베이스 개론

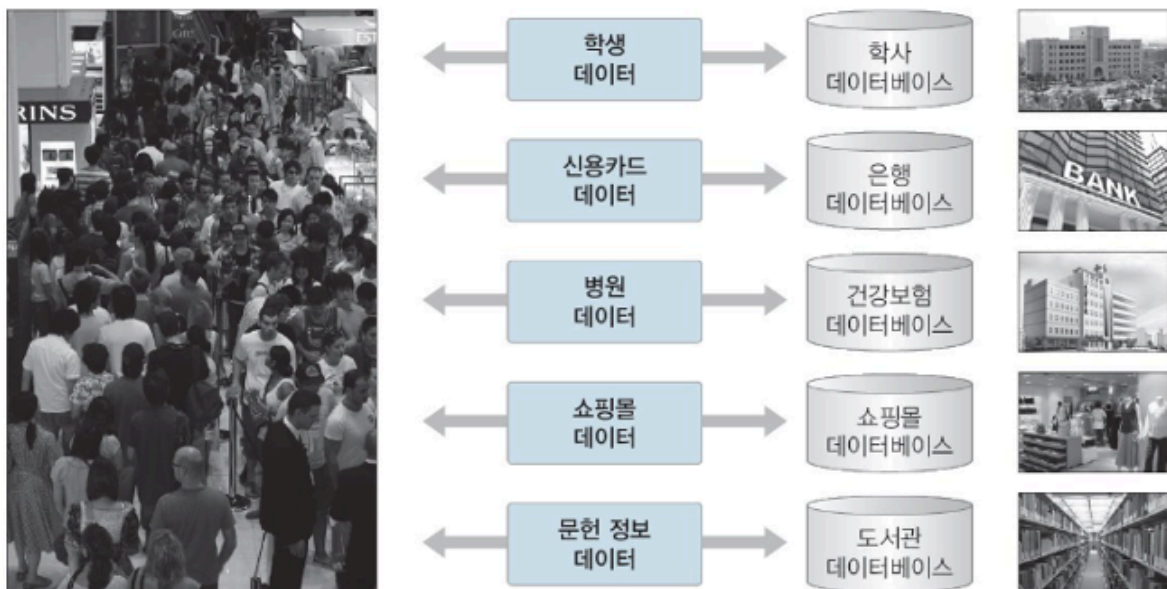
데이터, 정보, 지식의 관계

- 데이터 : 관찰의 결과로 나타난 정량적 혹은 정성적인 실제 값
- 정보 : 데이터에 의미를 부여한 것
- 지식 : 사물이나 현상에 대한 이해



데이터베이스란?

- 조직에 필요한 정보를 얻기 위해 논리적으로 연관된 데이터를 모아 구조적으로 통합해 놓은 것
- 여러 사용자나 응용 프로그램이 공유하고 동시에 접근 가능한 '데이터의 집합'
- '데이터의 저장 공간' 자체를 의미하기도 함

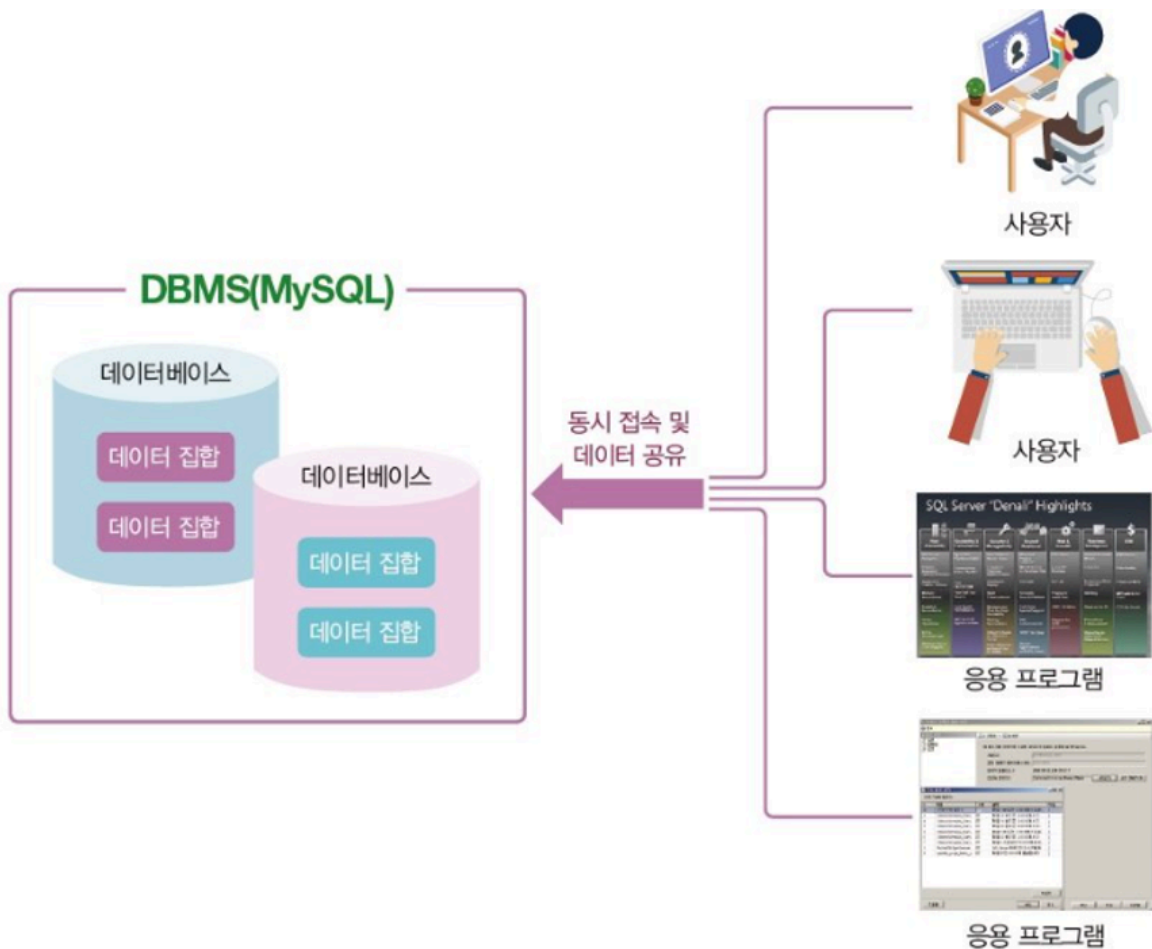


일상생활속 데이터베이스

DBMS	제작사	운영체제	최신 버전	비고
MySQL	오라클	유닉스, 리눅스, 윈도우, 맥	8.0	오픈 소스(무료), 상용
MariaDB	마리아DB	유닉스, 리눅스, 윈도우	10.3	오픈 소스(무료)
PostgreSQL	PostgreSQL	유닉스, 리눅스, 윈도우, 맥	10.4	오픈 소스(무료)
Oracle	오라클	유닉스, 리눅스, 윈도우	18c	상용 시장 점유율 1위
SQL Server	마이크로소프트	리눅스, 윈도우	2017	
DB2	IBM	유닉스, 리눅스, 윈도우	10	메인프레임 시장 점유율 1위
Access	마이크로소프트	윈도우	2017	PC용
SQLite	SQLite	안드로이드, iOS	3.24	모바일 전용, 오픈 소스(무료)

많이 사용되는 DBMS

데이터베이스 개념도



데이터베이스의 특징

- **데이터의 무결성** : 데이터베이스 안의 데이터는 어떤 경로를 통해 들어왔든 오류가 있어서는 안 됨
- **데이터의 독립성** : 데이터베이스와 응용 프로그램은 서로 의존적인 관계가 아니라 독립적인 관계임
- **보안**: 데이터베이스 안의 데이터는 데이터를 소유한 사람이나 데이터에 접근이 허가된 사람만 접근할 수 있음
- **데이터 중복 최소화** : 데이터베이스에서는 동일한 데이터가 여러 군데 중복 저장되는 것을 방지함
- **응용 프로그램 제작 및 수정 용이** : 데이터베이스를 이용하면 통일된 방식으로 응용 프로그램을 작성할 수 있고 유지·보수 또한 쉬움
- **데이터의 안전성 향상**: 데이터가 손상되는 문제가 발생하더라도 원래의 상태로 복원 또는 복구할 수 있음

관계형 DBMS

- 모든 데이터는 테이블에 저장
- 테이블 간의 관계는 **기본키(PK)**와 **외래키(FK)**를 사용하여 맺음(부모-자식 관계)
- 다른 DBMS에 비해 업무 변화에 따라 바로 순응할 수 있고 유지·보수 측면에서도 편리
- 대용량 데이터를 체계적으로 관리할 수 있음
- 데이터의 무결성도 잘 보장됨
- **시스템 자원을 많이 차지**하여 시스템이 전반적으로 느려지는 단점이 있음



릴레이션 (relation)

- 행과 열로 구성된 테이블

용어	한글 용어	비고
relation	릴레이션, 테이블	"관계"라고 하지 않음
relational data model	관계 데이터 모델	
relational database	관계 데이터베이스	
relational algebra	관계대수	
relationship	관계	

도서 1, 축구의 역사, 굿스포츠, 7000	도서번호	도서이름	출판사	가격
도서 2, 축구하는 여자, 나무수, 13000	1	축구의 역사	굿스포츠	7000
도서 3, 축구의 이해, 대한미디어, 22000	2	축구하는 여자	나무수	13000
도서 4, 골프 바이블, 대한미디어, 35000	3	축구의 이해	대한미디어	22000
도서 5, 피겨 교본, 굿스포츠, 8000	4	골프 바이블	대한미디어	35000
	5	피겨 교본	굿스포츠	8000

그림 2-1 데이터와 테이블(릴레이션)

도서번호 = {1, 2, 3, 4, 5}

도서이름 = {축구의 역사, 축구하는 여자, 축구의 이해, 골프 바이블, 피겨 교본}

출판사 = {굿스포츠, 나무수, 대한미디어}

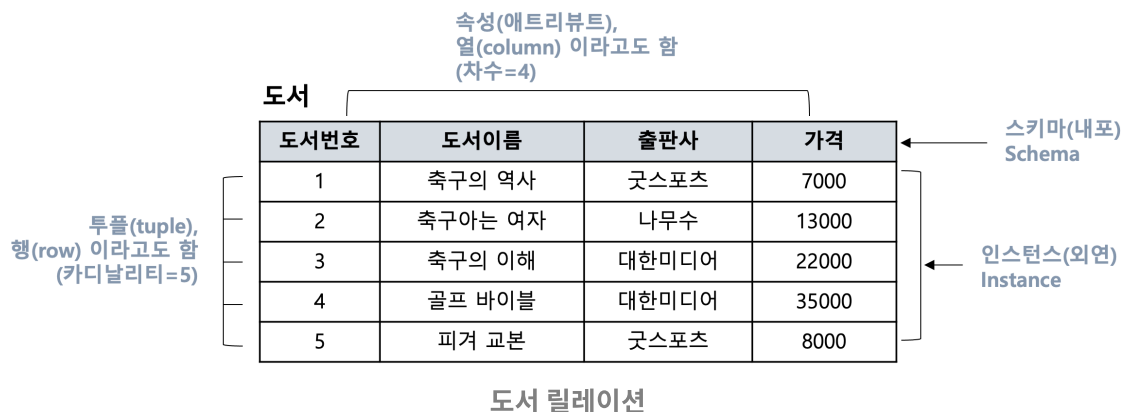
가격 = {7000, 13000, 22000, 35000, 8000}

→ 첫 번째 행(1, 축구의 역사, 굿스포츠, 7000)의 경우 네 개의 집합에서 각각 원소 한 개씩 선택하여 만들어진 것으로 이 원소들이 관계(relationship)를 맺고 있다.

- 관계(relationship)
 - 릴레이션 내에서 생성되는 관계 : 릴레이션 내 데이터들의 관계
 - 릴레이션 간에 생성되는 관계 : 릴레이션 간의 관계



릴레이션 스키마와 인스턴스



릴레이션 스키마

- 스키마의 요소
 - 속성(attribute) : 릴레이션 스키마의 열
 - 도메인(domain) : 속성이 가질 수 있는 값의 집합
 - 차수(degree) : 속성의 개수
- 스키마의 표현
 - 릴레이션 이름(속성1 : 도메인1, 속성2 : 도메인2, 속성3 : 도메인3 ...) EX) 도서(도서번호, 도서이름, 출판사, 가격)

릴레이션 인스턴스

- 인스턴스 요소
 - 튜플(tuple) : 릴레이션의 행
 - 카디널리티(cardinality) : 튜플의 수 (→ 튜플이 가지는 속성의 개수는 릴레이션 스키마의 차수와 동일하고, 릴레이션 내의 모든 튜플들은 서로 중복되지 않아야 함)

[릴레이션 구조와 관련된 용어]

릴레이션 용어	같은 의미로 통용되는 용어	파일 시스템 용어
릴레이션(relation)	테이블(table)	파일(file)
스키마(schema)	내포(intension)	헤더(header)
인스턴스(instance)	외연(extension)	데이터(data)
튜플(tuple)	행(row)	레코드(record)
속성(attribute)	열(column)	필드(field)

릴레이션의 특징

- 속성은 단일 값을 가진다
 - 각 속성의 값은 도메인에 정의된 값만을 가지며 그 값은 모두 단일 값이어야 함
- 속성은 서로 다른 이름을 가진다
 - 속성은 한 릴레이션에서 서로 다른 이름을 가져야만 함
- 한 속성의 값은 모두 같은 도메인 값을 가진다

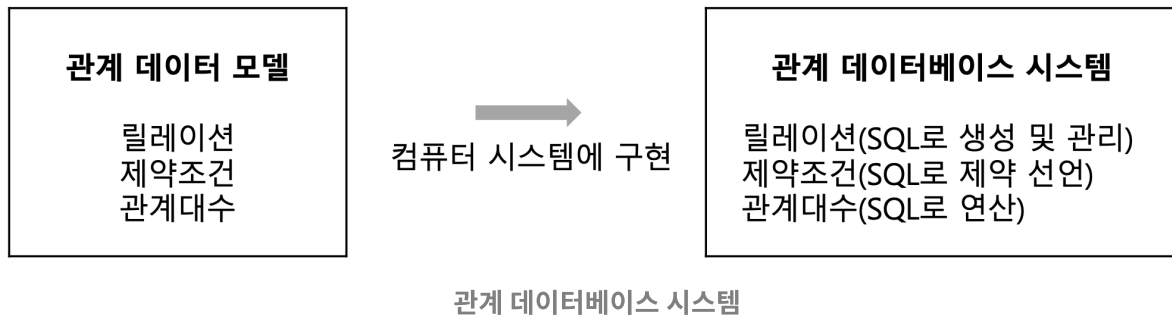
- 한 속성에 속한 열은 모두 그 속성에서 정의한 도메인 값만 가질 수 있음
- 속성의 순서는 상관없다
 - 속성의 순서가 달라도 릴레이션 스키마는 같음 예) 릴레이션 스키마에서 (이름, 주소) 순으로 속성을 표시하거나 (주소, 이름) 순으로 표시하여도 상관없음
- 릴레이션 내의 중복된 튜플은 허용하지 않는다
 - 하나의 릴레이션 인스턴스 내에서는 서로 중복된 값을 가질 수 없음, 즉 모든 튜플은 서로 값이 달라야 함
- 튜플의 순서는 상관없다
 - 튜플의 순서가 달라도 같은 릴레이션임. 관계 데이터 모델의 튜플은 실제적인 값을 가지고 있으며 이 값은 시간이 지남에 따라 데이터의 삭제, 수정, 삽입에 따라 순서가 바뀔 수 있음

[릴레이션의 특징에 위배된 경우]

도서번호	도서이름	출판사	가격
1	축구의 역사	굿스포츠	7000
2	축구 아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000
5	피겨 교본	굿스포츠	8000
6	피겨 교본, 피겨 기초	굿스포츠	8000

관계 데이터 모델

- 관계 데이터 모델은 데이터를 2차원 테이블 형태인 릴레이션으로 표현함
- 릴레이션에 대한 제약조건과 관계 연산을 위한 관계대수를 정의함



키(Key)

- 특정 튜플을 식별할 때 사용하는 속성 혹은 속성의 집합
- 릴레이션은 중복된 튜플을 허용하지 않음 → 각각의 튜플에 포함된 속성들 중 어느 하나 (혹은 하나 이상)는 값이 달라야 함. 즉 키가 되는 속성(혹은 속성의 집합)은 반드시 값이 달라서 튜플들을 서로 구별할 수 있어야 함
- 키는 릴레이션 간의 관계를 맺는 데도 사용됨

예

[고객]

고객번호	이름	주민번호	주소	핸드폰
1	박지성	810101-1111111	영국 맨체스타	000-5000-0001
2	김연아	900101-2222222	대한민국 서울	000-6000-0001
3	장미란	830101-2333333	대한민국 강원도	000-7000-0001
4	추신수	820101-1444444	미국 클리블랜드	000-8000-0001

[도서]

도서번호	도서이름	출판사	가격
1	축구의 역사	굿스포츠	7000
2	축구아는 여자	나무수	13000
3	축구의 이해	대한미디어	22000

도서번호	도서이름	출판사	가격
4	골프 바이블	대한미디어	35000
5	피겨 교본	굿스포츠	8000

[주문]

고객번호	도서번호	판매가격	주문일자
1	1	7000	2014-07-01
1	2	13000	2014-07-03
2	5	8000	2014-07-03
3	2	13000	2014-07-04
4	4	35000	2014-07-05
1	3	22000	2014-07-07
4	3	22000	2014-07-07

기본키

- 여러 후보키 중 하나를 선정하여 대표로 삼는 키
- 후보키가 하나뿐이라면 그 후보키를 기본키로 사용하면 되고, 여러 개라면 릴레이션의 특성을 반영하여 하나를 선택하면 됨
- 기본키 선정 시 고려사항
 - 릴레이션 내 튜플을 식별할 수 있는 고유한 값을 가져야 함.
 - NULL 값은 허용하지 않음.
 - 키 값의 변동이 일어나지 않아야 함.
 - 최대한 적은 수의 속성을 가진 것이라야 함.
 - 향후 키를 사용하는 데 있어서 문제 발생 소지가 없어야 함.
- 릴레이션 스키마를 표현할 때 기본키는 밑줄을 그어 표시함
 - 릴레이션 이름(속성1, 속성2, 속성N)
 - EX) 고객(고객번호, 이름, 주민번호, 주소, 핸드폰)
 - 도서(도서번호, 도서이름, 출판사, 가격)

외래키

- 다른 릴레이션의 기본키를 참조하는 속성을 말함
- 다른 릴레이션의 기본키를 참조하여 관계 데이터 모델의 특징인 릴레이션 간의 관계 (relationship)를 표현함
- 외래키의 특징
 - 관계 데이터 모델의 릴레이션 간의 관계를 표현함
 - 다른 릴레이션의 기본키를 참조하는 속성임
 - 참조하고(외래키) 참조되는(기본키) 양쪽 릴레이션의 도메인은 서로 같아야 함
 - 참조되는(기본키) 값이 변경되면 참조하는(외래키) 값도 변경됨
 - NULL 값과 중복 값 등이 허용됨
 - 자기 자신의 기본키를 참조하는 외래키도 가능함
 - 외래키가 기본키의 일부가 될 수 있음

무결성 제약조건

- **데이터 무결성(integrity, 無缺性)**
 - 데이터베이스에 저장된 데이터의 일관성과 정확성을 지키는 것
- **도메인 무결성 제약조건**
 - 도메인 제약(domain constraint)이라고도 하며, 릴레이션 내의 튜플들이 각 속성의 도메인에 지정된 값만을 가져야 한다는 조건. SQL 문에서 데이터 형식(type), 널(null/not null), 기본 값(default), 체크(check) 등을 사용하여 지정할 수 있음.
- **개체 무결성 제약조건**
 - 기본키 제약(primary key constraint)이라고도 함. 릴레이션은 기본키를 지정하고 그에 따른 무결성 원칙, 즉 기본키는 NULL 값을 가져서는 안 되며 릴레이션 내에 오직 하나의 값만 존재해야 한다는 조건임.
- **참조 무결성 제약조건**
 - 외래키 제약(foreign key constraint)이라고도 하며, 릴레이션 간의 참조 관계를 선언하는 제약조건임. 자식 릴레이션의 외래키는 부모 릴레이션의 기본키와 도메인이 동일해야 하며, 자식 릴레이션의 값이 변경될 때 부모 릴레이션의 제약을 받는다는 것임
- **개체 무결성 제약조건**
 - 삽입 : 기본키 값이 같으면 삽입이 금지됨

- 수정 : 기본키 값이 같거나 NULL로도 수정이 금지됨
- 삭제 : 특별한 확인이 필요하지 않으며 즉시 수행함

(501, 남슬찬, 1001)

↓ 삽입 거부

학번	이름	학과코드
501	박지성	1001
401	김연아	2001
402	장미란	2001
502	추신수	1001

(NULL, 남슬찬, 1001)

↓ 삽입 거부

학번	이름	학과코드
501	박지성	1001
401	김연아	2001
402	장미란	2001
502	추신수	1001

그림 학생 릴레이션

그림 개체 무결성 제약조건 수행 예
(기본키 충돌 및 NULL 값 삽입)

참조 무결성 제약조건

- 삽입
 - 학과(부모 릴레이션) : 튜플 삽입한 후 수행하면 정상적으로 진행된다.
 - 학생(자식 릴레이션) : 참조받는 테이블에 외래키 값이 없으므로 삽입이 금지된다.

학생(자식 릴레이션)

학번	이름	학과코드
501	박지성	1001
401	김연아	2001
402	장미란	2001
502	추신수	1001

학과(부모 릴레이션)

학과코드	학과명
1001	컴퓨터학과
2001	체육학과

참조

그림 학생관리 데이터베이스

- 삭제
 - 학과(부모 릴레이션) : 참조하는 테이블을 같이 삭제할 수 있어서 금지하거나 다른 추가 작업 필요
 - 학생(자식 릴레이션) : 바로 삭제 가능함

※ 부모 릴레이션에서 튜플을 삭제할 경우 참조 무결성 조건을 수행하기 위한 고려 사항

- 즉시 작업을 중지
- 자식 릴레이션의 관련 튜플을 삭제
- 초기에 설정된 다른 어떤 값으로 변경
- NULL 값으로 설정

◦ 수정

- 삭제와 삽입 명령이 연속해서 수행됨.
- 부모 릴레이션의 수정이 일어날 경우 삭제 옵션에 따라 처리된 후 문제가 없으면 다시 삽입 제약조건에 따라 처리됨.

SQL

SQL이란?

- Structured Query Language(구조화된 질의 언어)
- NCITS(국제표준화위원회)에서 ANSI/ISO SQL이라는 명칭의 SQL 표준을 관리하고 있음
- 1992년에 제정된 ANSI-92 SQL과 1999년에 제정된 ANSI-99 SQL을 대부분의 DBMS 회사에서 SQL 표준으로 사용하고 있음
- 각 회사는 ANSI-92/99 SQL의 표준을 준수하면서도 자신의 제품 특성을 반영한 SQL에 별도의 이름을 붙임: MySQL에서는 그냥 SQL, 오라클에서는 PL/SQL, SQL Server에서는 Transact SQL(T-SQL) 사용
- 역할에 따라 데이터 정의어(DDL, Data Definition Language), 데이터 조작어(DML, Data Manipulation Language), 데이터 제어어(DCL, Data Control Language)로 분류됨
- 데이터베이스 관리 시스템(DBMS, Database Management System)의 제조사에 따라 조금씩 다른 문법의 SQL로 데이터베이스 관리 가능

MySQL 개요

- 오라클에서 제작한 DBMS 소프트웨어
- 오픈소스로 제공됨
- 기업 사용자의 경우 라이선스 비용이 발생하나, 개인 사용자는 무료로 사용 가능 (Community Edition)

시기	버전	기타
1994년		마이클 와이드나우스, 데이비드 엑스마크가 개발
1995년 5월		최초 국제 버전 공개
1996년 말	3.19	
1997년 1월	3.20	
1998년 1월		윈도우용 릴리스
1998년	3.21	상용 제품 릴리스, www.mysql.com 사이트 구축
2003년 3월	4.0	
2004년 10월	4.1	R-트리, B-트리, 서브쿼리 등 지원
2005년 10월	5.0	커서, 스토어드 프로시저, 트리거, 뷰 등 지원
2008년		선마이크로시스템에 인수됨
2008년 11월	5.1	픽티셔닝, 복제 등 지원
2010년 1월		오라클이 선마이크로시스템을 인수
2010년 12월	5.5	InnoDB를 기본 엔진으로 사용하는 등 큰 변화가 있었음
2013년 2월	5.6	성능 개선, NoSQL 지원 등
2015년 10월	5.7	보안 강화, InnoDB 확장, JSON 지원 등
2018년 4월	8.0	성능 향상, 보안 강화, 장애 대비 능력 강화 등

MySQL 설치 준비

- 내 컴퓨터 운영체제 확인

서버 운영체제(64bit)	PC 운영체제(64bit)
Windows Server 2019	윈도우 10
Windows Server 2016	윈도우 7(공식적으로는 지원하지 않음)
Windows Server 2012 R2	

- 설치 프로그램 소개(초록색으로 표시된 것만 설치)

관련 소프트웨어	요구 사항	요구 사항 설치	비고
MySQL Server	Visual C++ 2015 Runtime	자동	샤어 프로그램
MySQL Client			클라이언트 프로그램
MySQL Workbench	Visual C++ 2015 Runtime	자동	GUI 지원 통합 관리 툴
Sample Database			샘플 데이터베이스
MySQL Notifier			샤어스 알림 기능

- MySQL 설치 동영상: <https://www.youtube.com/watch?v=vAy4079h4yU&t=18s>
- MySQL Community 다운로드하기 – 윈도우의 경우 MSI installer로 다운로드
<https://dev.mysql.com/downloads/mysql/>

MySQL Community Downloads

MySQL Community Server

General Availability (GA) Releases Archives

MySQL Community Server 8.3.0 Innovation

Select Version:
8.3.0 Innovation

Select Operating System:
Microsoft Windows

Operating System	Version	Size	Download
Windows (x86, 64-bit), MSI Installer (mysql-8.3.0-winx64.msi)	8.3.0	135.1M	Download
Windows (x86, 64-bit), ZIP Archive (mysql-8.3.0-winx64.zip)	8.3.0	257.9M	Download
Windows (x86, 64-bit), ZIP Archive Debug Binaries & Test Suite (mysql-8.3.0-winx64-debug-test.zip)	8.3.0	725.8M	Download

We suggest that you use the MD5 checksums and GnuPG signatures to verify the integrity of the packages you download.

MySQL Community Downloads

Login Now or Sign Up for a free account.

An Oracle Web Account provides you with the following advantages:

- Fast access to MySQL software downloads
- Download technical White Papers and Presentations
- Post messages in the MySQL Discussion Forums
- Report and track bugs in the MySQL bug system

Login »
using my Oracle Web account

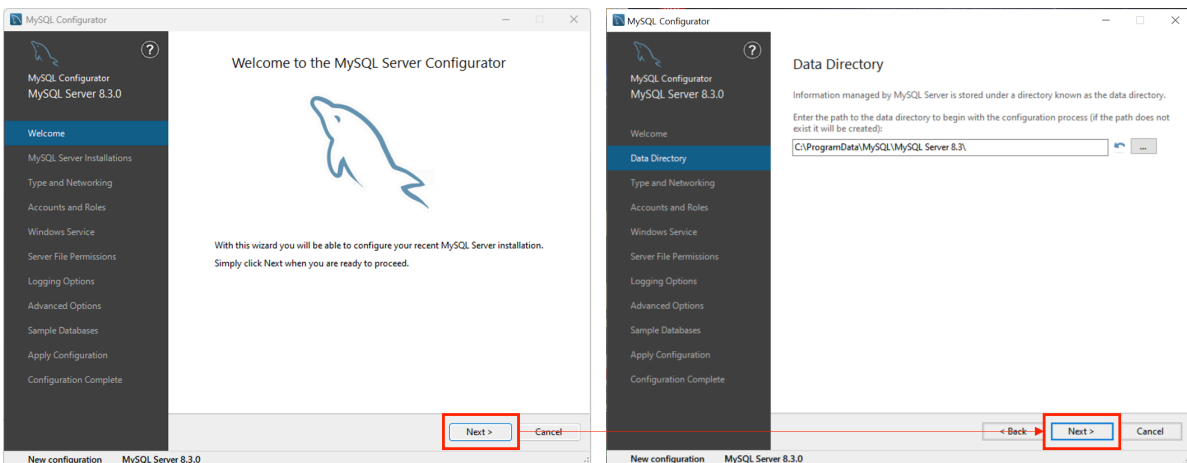
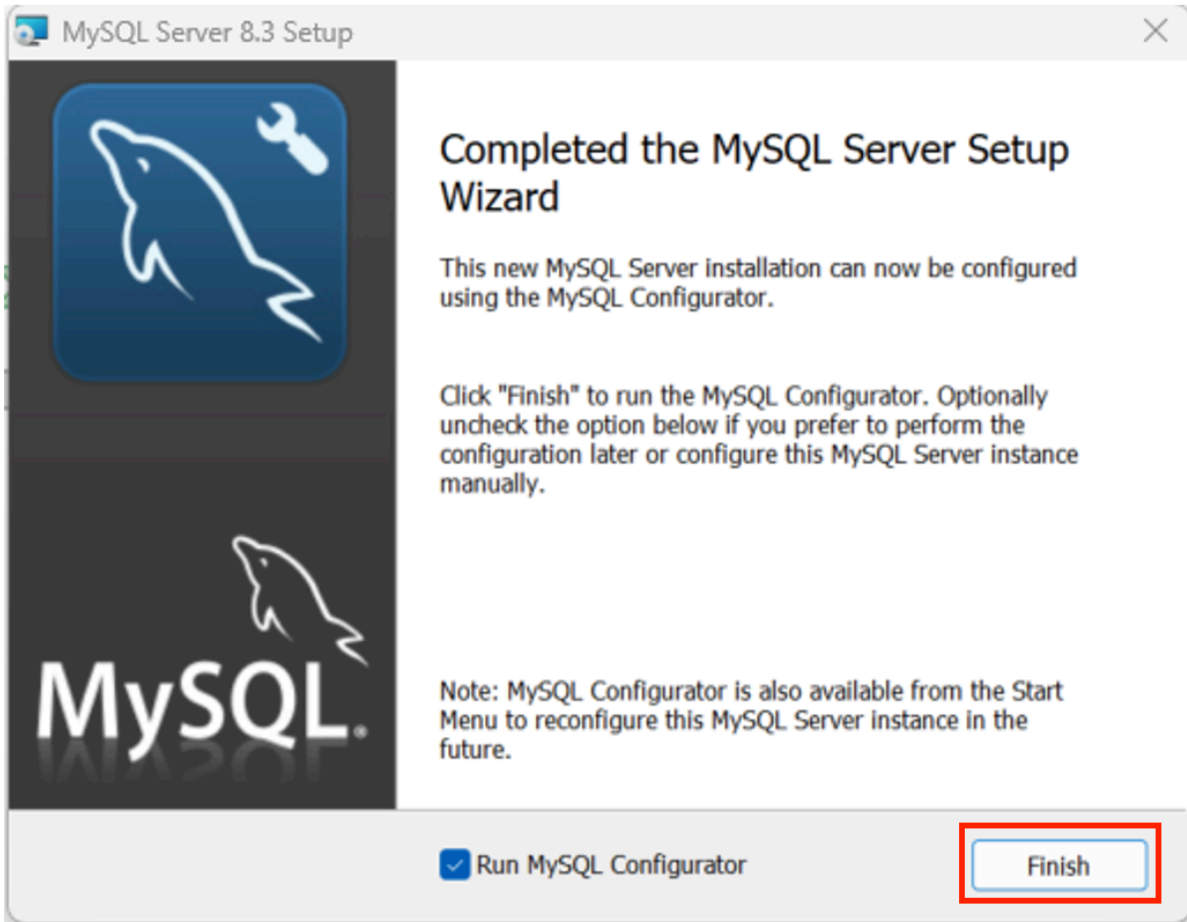
Sign Up »
for an Oracle Web account

MySQL.com is using Oracle SSO for authentication. If you already have an Oracle Web account, click the Login link. Otherwise, you can sign up for a free account by clicking the Sign Up link and following the instructions.

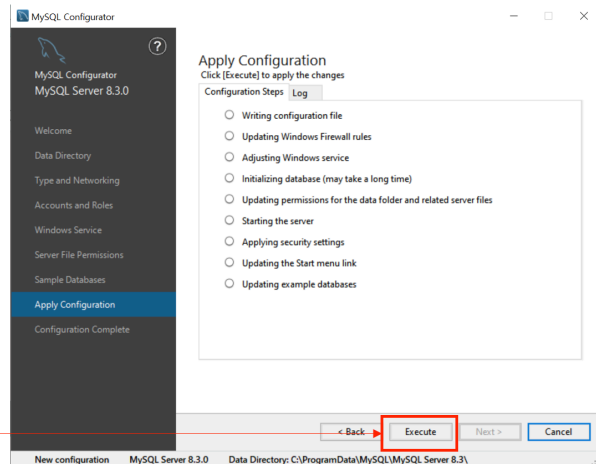
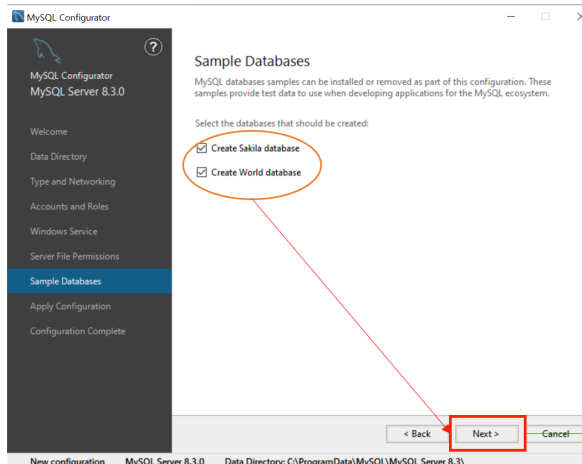
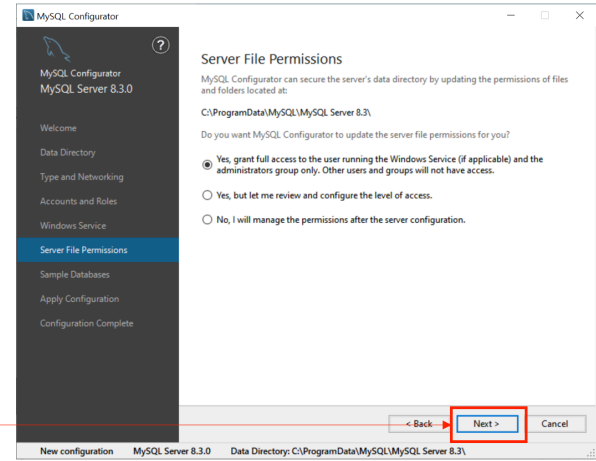
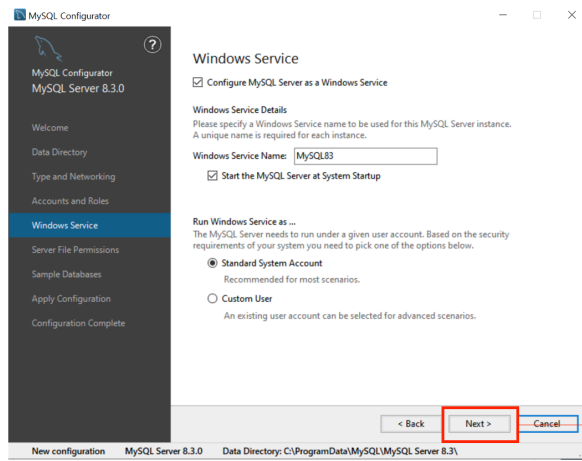
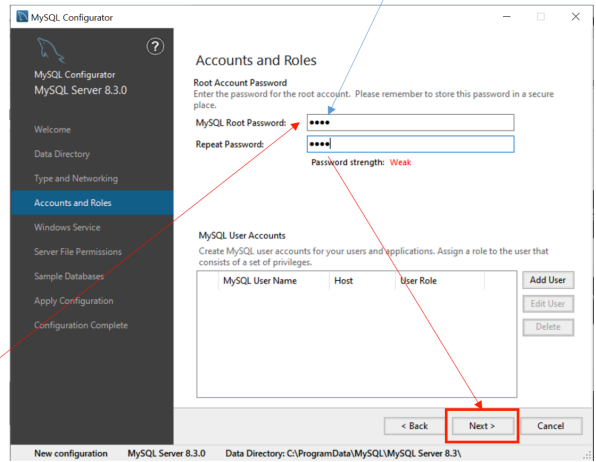
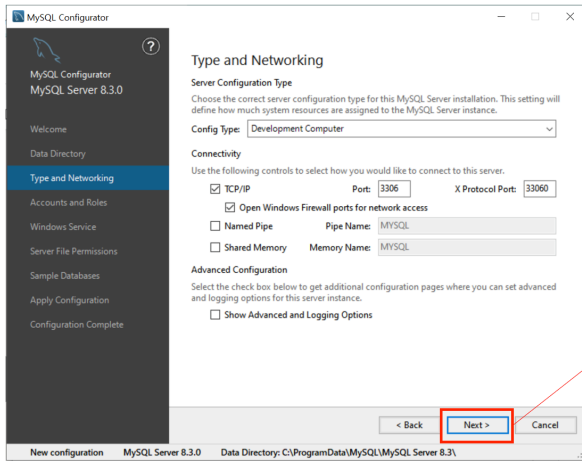
No thanks, just start my download.

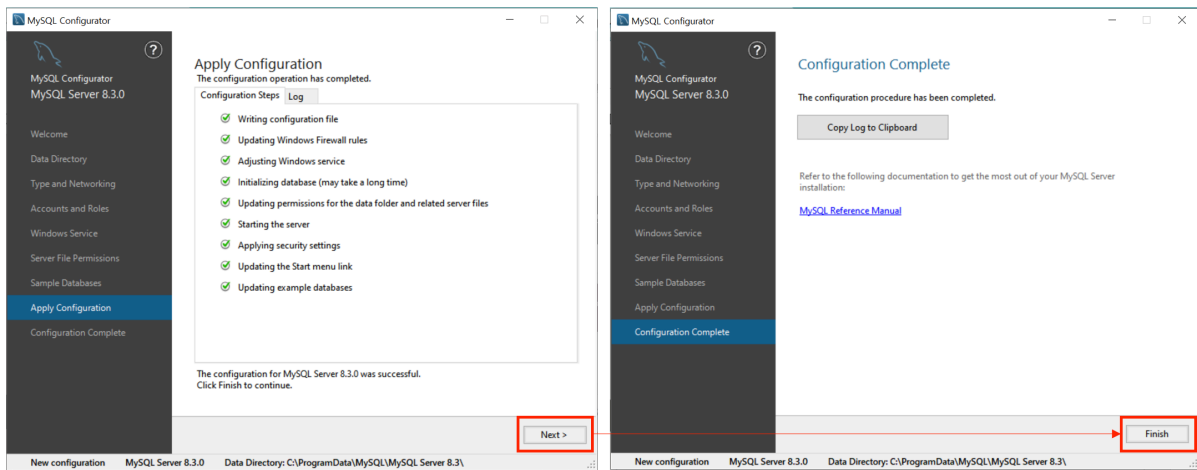
- 다운로드한 MySQL 설치하기





- 1.임의의 비밀번호 입력
- 2.우측의 Connect 버튼 클릭

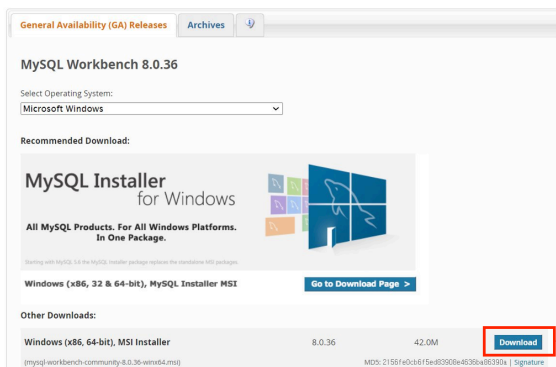




- Workbench 다운로드 - <https://dev.mysql.com/downloads/workbench/>
- 맥북에서 Workbench 다운로드 설치 과정
<https://www.codeit.kr/tutorials/12/MySQL-Workbench-설치-macOS/>

MySQL Community Downloads

MySQL Workbench

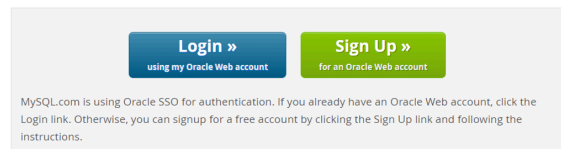


MySQL Community Downloads

Login Now or Sign Up for a free account.

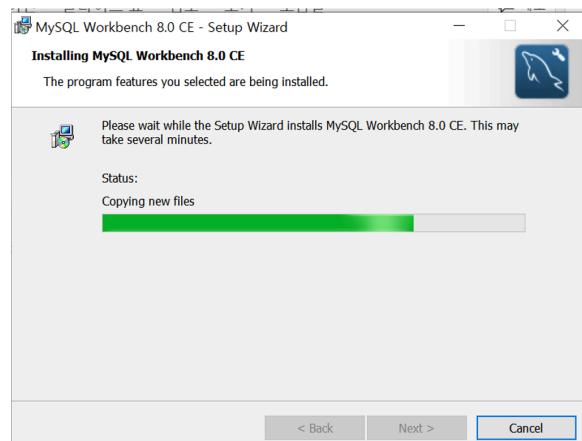
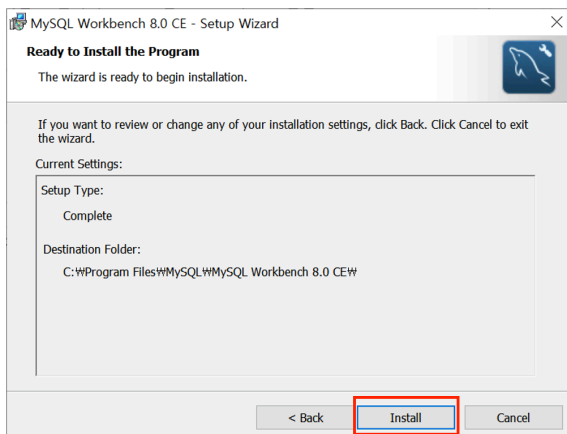
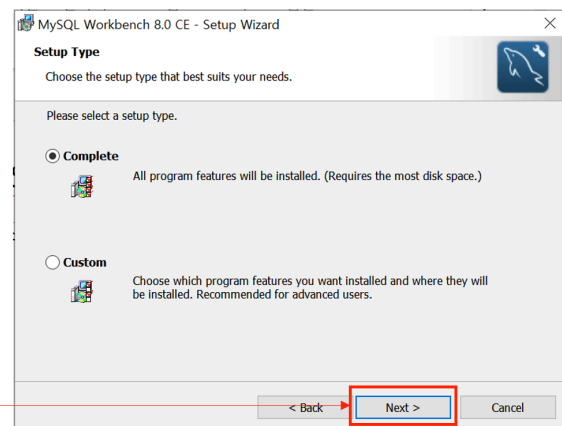
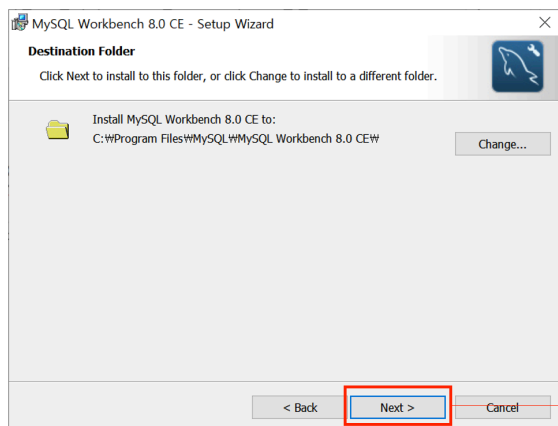
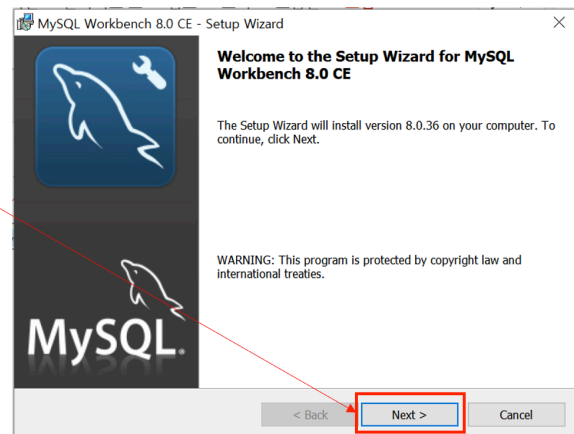
An Oracle Web Account provides you with the following advantages:

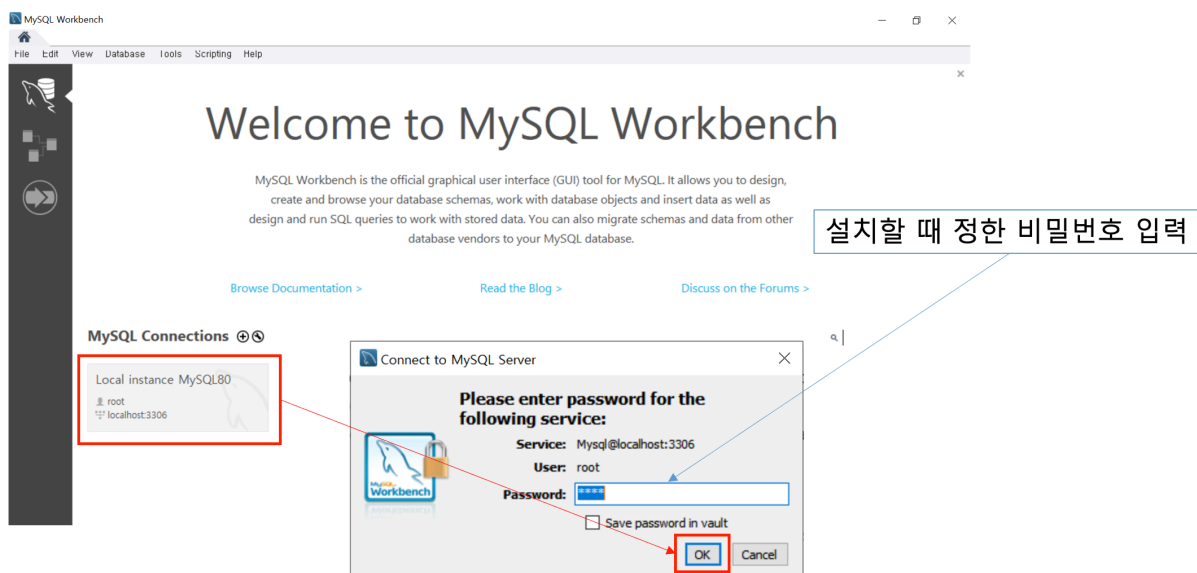
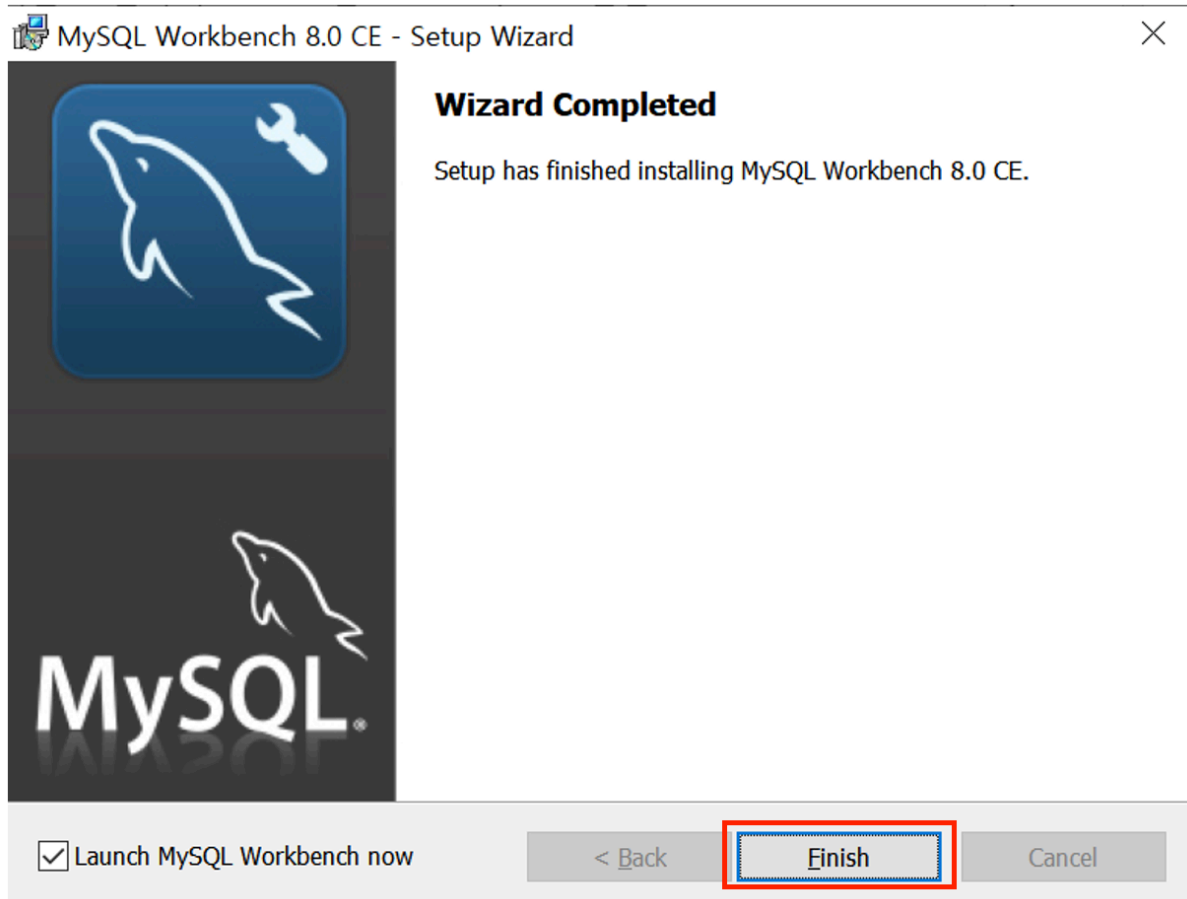
- Fast access to MySQL software downloads
- Download technical White Papers and Presentations
- Post messages in the MySQL Discussion Forums
- Report and track bugs in the MySQL bug system

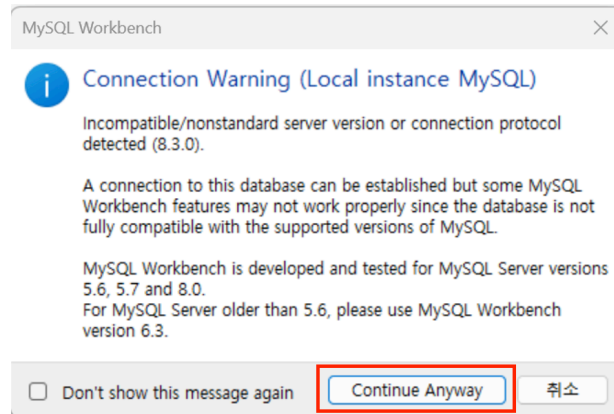


No thanks, just start my download.

mysql-workbench-community-8.0.36-winx64





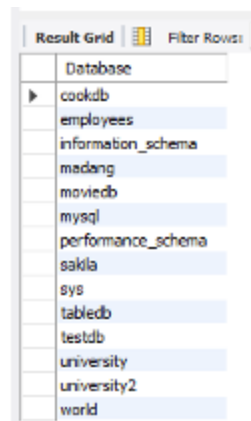


데이터베이스 이름 조회하기

- 현재 MySQL 서버에 어떤 데이터베이스가 있는지 잘 모를 때, 다음과 같이 입력 후 실행

```
SHOW DATABASES;
```

- 실행하고자 하는 쿼리 입력 후, 해당 라인(line)만 마우스로 블록구간을 잡은 후 상단의 번개 표시 아이콘을 클릭하여 실행:
-



- USE
 - 현재 조회/수정/삭제하고자 하는 데이터가 있는 데이터베이스를 지정 혹은 변경하고자 할 때 사용하는 구문 형식

```
USE 데이터베이스이름;
```

- 만약 sakila 데이터베이스를 사용하려면 다음과 같이 입력

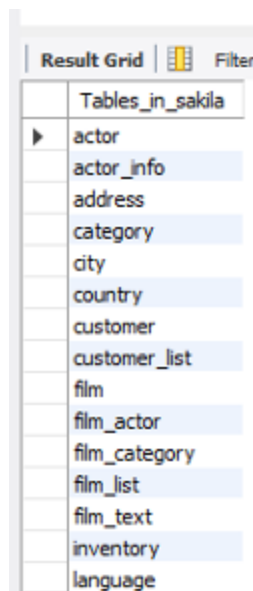
```
USE sakila;
```

- 작업할 데이터베이스가 선택되어 있어야만 쿼리 수행 가능

- 테이블 이름 조회하기

- 현재 데이터베이스에 어떤 테이블이 있는지 조회

```
SHOW TABLES;
```



Tables_in_sakila	
▶	actor
	actor_info
	address
	category
	city
	country
	customer
	customer_list
	film
	film_actor
	film_category
	film_list
	film_text
	inventory
	language

- 테이블 내의 열 자세히 조회하기

- 테이블 film의 열에는 무엇이 있는지 확인

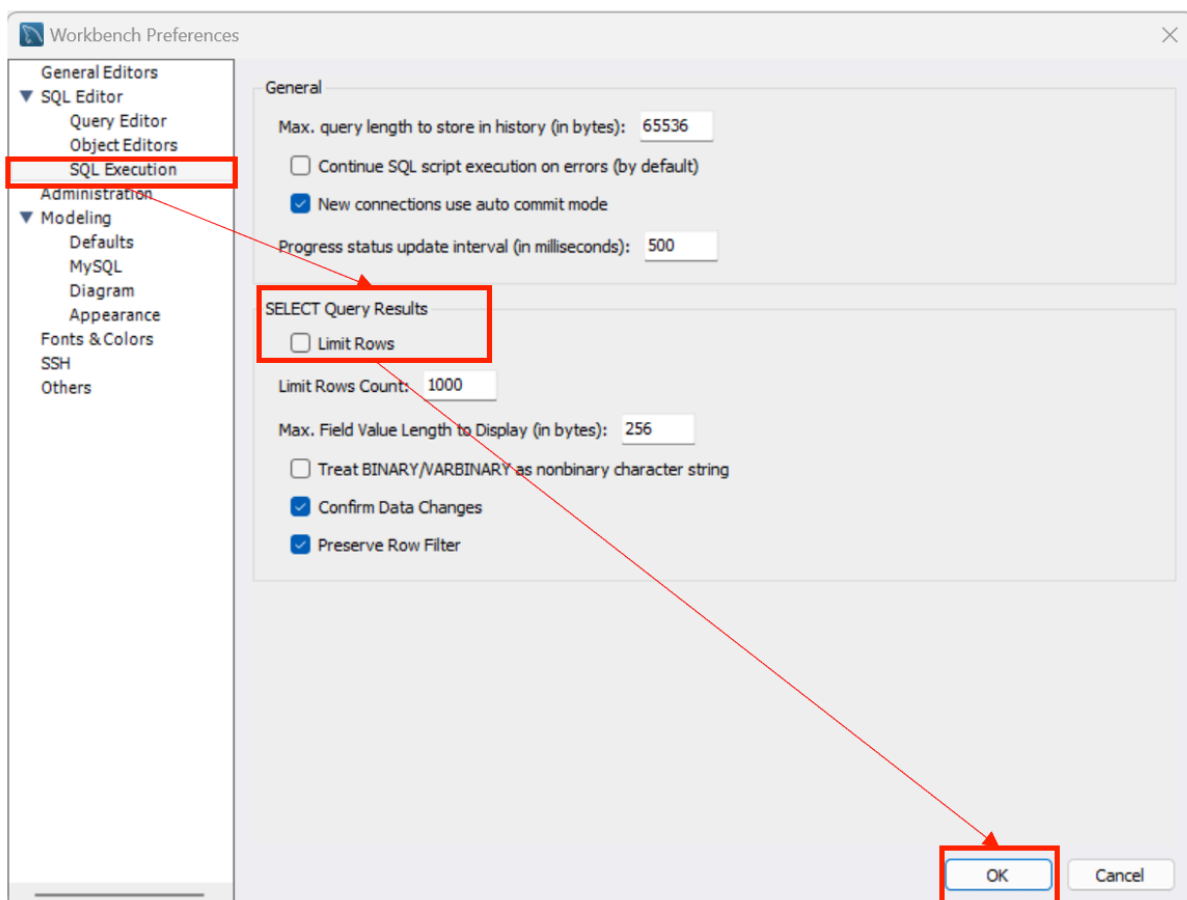
```
DESCRIBE film;
```

또는

```
DESC film;
```

Field	Type	Null	Key	Default	Extra
film_id	smallint(5) unsigned	NO	PRI	NULL	auto_increment
title	varchar(255)	NO	MUL	NULL	
description	text	YES		NULL	
release_year	year(4)	YES		NULL	
language_id	tinyint(3) unsigned	NO	MUL	NULL	
original_language_id	tinyint(3) unsigned	YES	MUL	NULL	
rental_duration	tinyint(3) unsigned	NO		3	
rental_rate	decimal(4,2)	NO		4.99	
length	smallint(5) unsigned	YES		NULL	
replacement_cost	decimal(5,2)	NO		19.99	
rating	enum('G','PG','PG-1...	YES		G	
special_features	set('Trailers','Com...	YES		NULL	
last_update	timestamp	NO		CURR...	DEFAULT_GEN...

- 전체 조회를 위한 준비
 - 상단 메뉴의 [Edit]-[Preferences] 클릭
 - 새 팝업의 좌측에서 [SQL Editor]-[SQL Execution] 클릭
 - 우측의 'SELECT Query Results'에서 바로 하단의 'Limit Rows' 체크 해제
 - 하단의 'OK' 버튼 눌러서 설정 반영



- SELECT

- 데이터베이스에서 원하는 값을 조회하고자 할 때 사용하는 쿼리
- 데이터 조작어(DML)에 해당함
- 사용 문법

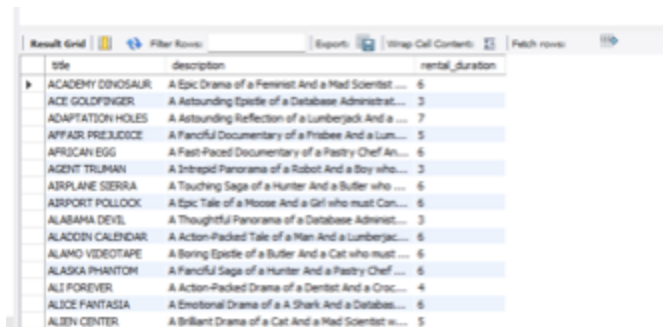
```
SELECT 열이름  
FROM 테이블이름  
WHERE 조건
```

- SELECT ... FROM 문
- 테이블 film의 모든 열(column) 검색

```
SELECT * FROM film;
```

- 특정한 열만 가져오고 싶으면 열의 이름을 기재하고 쉼표(,)로 구분

```
SELECT title, description, rental_duration from film;
```



	title	description	rental_duration
▶	ACADEMY DINOSAUR	A Epic Drama of a Feminist And a Mad Scientist ...	6
	ACE GOLDFINGER	A Astounding Epistle of a Database Administrat...	3
	ADAPTATION HOLES	A Astounding Reflection of a Lumberjack And a ...	7
	AFFAIR PREJUDICE	A Painful Documentary of a Pinbee And a Lum...	5
	AFRICAN EGG	A Fast-Paced Documentary of a Pastry Chef An...	6
	AGENT TRUMAN	A Intrepid Panorama of a Robot And a Boy who...	3
	AIRPLANE SIERRA	A Touching Saga of a Hunter And a Butler who ...	6
	AIRPORT POLLOCK	A Epic Tale of a Moose And a Girl who must Con...	6
	ALABAMA DEVIL	A Thoughtful Panorama of a Database Administr...	3
	ALADDIN CALENDAR	A Action-Packed Tale of a Man And a Lumberjac...	6
	ALAMO VIDEOTAPE	A Boring Epistle of a Butler And a Cat who must ...	6
	ALASKA PHANTOM	A Painful Saga of a Hunter And a Pastry Chef ...	6
	ALI FOREVER	A Action-Packed Drama of a Dentist And a Crac...	4
	ALICE FANTASIA	A Emotional Drama of a A Shark And a Databas...	6
	ALIEN CENTER	A Brilliant Drama of a Cat And a Mad Scientist e...	5

- 정해진 수의 결과만 가져오고 싶다면 맨 뒤에 LIMIT 붙여 조회

```
SELECT title, description, rental_duration from film LIMIT 10;
```

- SELECT ... FROM 문에 WHERE 절을 추가하면 특정한 조건을 만족하는 데이터만 조회할 수 있음

SELECT 열이름 FROM 테이블이름 WHERE 조건식;

- WHERE 절을 추가하면 원하는 데이터만 조회 가능

SELECT * FROM film WHERE rental_duration=6;

film_id	title	description	release_year	language_id	original_language_id	rental_duration	rental_rate	length	replacement_cost	rating	special_features	last_update
1	ACADEMY DINOSAUR	A Epic Drama of a Feminist And a Mad Scientist...	2006	1	1	6	0.99	86	20.99	PG	Deleted Scenes,Behind the Scenes	2006-02-15 05:03:42
5	AFRICAN EGG	A Fast-Paced Documentary of a Pastry Chef And...	2006	1	1	6	2.99	130	22.99	G	Deleted Scenes	2006-02-15 05:03:42
7	AIRPLANE SIERRA	A Touching Saga of a Hunter And a Butler who...	2006	1	1	6	4.99	62	28.99	PG-13	Trailers,Deleted Scenes	2006-02-15 05:03:42
8	AIRPORT POLLOCK	A Epic Tale of a Moose And a Girl who must Con...	2006	1	1	6	4.99	54	15.99	R	Trailers	2006-02-15 05:03:42
10	ALADDIN CALENDAR	A Action-Packed Tale of a Man And a Lumberjac...	2006	1	1	6	4.99	63	24.99	NC-17	Trailers,Deleted Scenes	2006-02-15 05:03:42
11	ALAMO VIDEOTAPE	A Boring Epistle of a Butler And a Cat who must...	2006	1	1	6	0.99	126	16.99	G	Commentaries,Behind the Scenes	2006-02-15 05:03:42
12	ALASKA PHANTOM	A Fanciful Saga of a Hunter And a Pastry Chef...	2006	1	1	6	0.99	136	22.99	PG	Commentaries,Deleted Scenes	2006-02-15 05:03:42
14	ALICE FANTASIA	A Emotional Drama of a a Shark And a Databab...	2006	1	1	6	0.99	94	23.99	NC-17	Trailers,Deleted Scenes,Behind the Scenes	2006-02-15 05:03:42
16	ALLEY EVOLUTION	A Fast-Paced Drama of a Robot And a Composer...	2006	1	1	6	2.99	180	23.99	NC-17	Trailers,Commentaries	2006-02-15 05:03:42
18	ALTER VICTORY	A Thoughtful Drama of a Composer And a Femi...	2006	1	1	6	0.99	57	27.99	PG-13	Trailers,Behind the Scenes	2006-02-15 05:03:42
19	AMADEUS HOLY	A Emotional Display of a Pioneer And a Technica...	2006	1	1	6	0.99	113	20.99	PG	Commentaries,Deleted Scenes,Behind the...	2006-02-15 05:03:42
22	AMISTAD MIDSUMMER	A Emotional Character Study of a Dentist And a...	2006	1	1	6	2.99	86	10.99	G	Commentaries,Behind the Scenes	2006-02-15 05:03:42
24	ANALYZE HOOSIERS	A Thoughtful Display of a Explorer And a Pastry...	2006	1	1	6	2.99	181	19.99	R	Trailers,Behind the Scenes	2006-02-15 05:03:42
32	APOCALYPSE FLAME...	A Astonishing Story of a Dog And a Squirrel who...	2006	1	1	6	4.99	119	11.99	R	Trailers,Commentaries	2006-02-15 05:03:42
34	ARABIA DIGHA	A Touching Epistle of a Madman And a Mad Com...	2006	1	1	6	0.99	62	29.99	NC-17	Commentaries,Deleted Scenes	2006-02-15 05:03:42

- SELECT – AND/OR

- 조건 연산자와 관계 연산자: WHERE 절 안의 조건을 여러 개를 줄 수 있음

SELECT * FROM film WHERE rental_duration=6 AND rental_rate=0.99;
 SELECT * FROM film WHERE rental_duration=6 OR rental_rate=0.99;
 SELECT * FROM film WHERE rental_duration=6 AND rental_rate<=2.99;

- SELECT – BETWEEN ... AND

SELECT title, description, rating FROM film WHERE rental_rate>=0.99 A
 ND rental_rate<=2.99;
 SELECT title, description, rating FROM film WHERE rental_rate BETWEE
 N 0.99 AND 2.99;

- SELECT - IN

SELECT title, description, rating FROM film WHERE rating='PG' OR ratin
 g='G';
 SELECT title, description, rating FROM film WHERE rating IN('PG','G');

- 위 두 개의 쿼리는 같은 기능
- IN을 사용하는 것이 쿼리 길이가 조금 짧음

참고 : PG(Parental Guidance suggested), G(General audiences) -
https://en.wikipedia.org/wiki/Motion_Picture_Association_film_rating_system

- SELECT - NOT IN

- 첫 번째 쿼리의 결과 중에서 두 번째 쿼리에 해당하는 것을 제외하고 출력

```
SELECT * FROM film WHERE rating NOT IN ('G', 'PG');
```

- 첫 번째 쿼리의 결과 중에서 두 번째 쿼리에 해당하는 것을 제외하고 출력

```
SELECT * FROM film WHERE rating NOT IN (SELECT rating FROM film  
WHERE rating='PG');
```

- SELECT - LIKE

- DINOSAUR로 끝나는 문자열을 제목으로 가진 row만 결과로 가져오기

```
SELECT title, description, rating FROM film WHERE title LIKE '%DINOSA  
UR';
```

- HUNT로 시작하는 문자열을 제목으로 가진 row만 결과로 가져오기

```
SELECT title, description, rating FROM film WHERE title LIKE 'HUNT%';
```

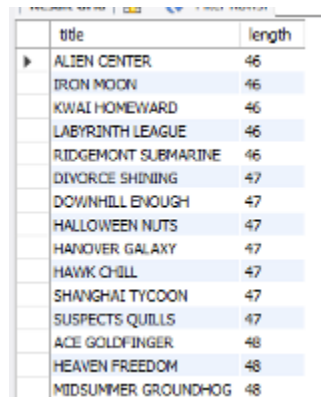
- 서브쿼리 사용으로 특정 데이터와 비교하여 만족하는 결과만 가져오기

```
SELECT title, length FROM film  
WHERE length>=(SELECT length FROM film WHERE title='INTRIGUE WOR  
ST');
```

- SELECT – ORDER BY

- 특정 열을 기준으로 순서를 정하여 출력하기
- 기본적으로 오름차순(ascending) 정렬

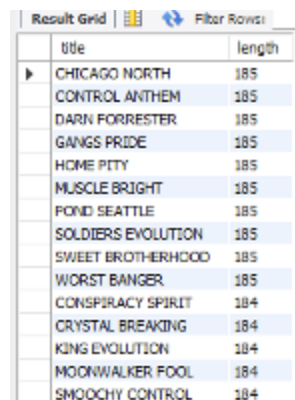

```
SELECT title, length FROM film ORDER BY length;
```



title	length
ALIEN CENTER	46
IRON MOON	46
KWAI HOMEWARD	46
LABYRINTH LEAGUE	46
RIDGEMONT SUBMARINE	46
DIVORCE SHINING	47
DOWNHILL ENOUGH	47
HALLOWEEN NUTS	47
HANOVER GALAXY	47
HAWK CHILL	47
SHANGHAI TYCOON	47
SUSPECTS QUILLS	47
ACE GOLDFINGER	48
HEAVEN FREEDOM	48
MIDSUMMER GROUNDHOG	48

- 내림차순(descending)으로 바꾸어 보고 싶으면 뒤에 **DESC** 붙여서 정렬

```
SELECT title, length FROM film ORDER BY length DESC;
```



title	length
CHICAGO NORTH	185
CONTROL ANTHEM	185
DARN FORRESTER	185
GANGS PRIDE	185
HOME PITY	185
MUSCLE BRIGHT	185
POND SEATTLE	185
SOLDIERS EVOLUTION	185
SWEET BROTHERHOOD	185
WORST BANGER	185
CONSPIRACY SPIRIT	184
CRYSTAL BREAKING	184
KING EVOLUTION	184
MOONWALKER FOOL	184
SMOOCY CONTROL	184

- 정렬 기준을 2개로 설정하는 것도 가능 - ASC는 생략 가능

```
SELECT title, length FROM film ORDER BY length DESC, title ASC;
```

- SELECT - DISTINCT
 - 겹치지 않는 결과만 가져오도록 쿼리 실행하기

```
SELECT DISTINCT rating FROM film;
```

	rating
▶	PG
	G
	NC-17
	PG-13
	R

- 집계함수 (참고)

함수	설명
AVG()	평균을 구한다.
MIN()	최솟값을 구한다.
MAX()	최댓값을 구한다.
COUNT()	행의 개수를 센다.
COUNT(DISTINCT)	행의 개수를 센다(중복은 1개만 인정).
STDEV()	표준편차를 구한다.
VAR_SAMP()	분산을 구한다.

- SELECT – GROUP BY
 - 영화 등급별로 몇 개의 영화가 있는지 확인

COUNT 대상이 되는 열 이름을 반드시 지정해야 의도대로 결과가 나옴
 SELECT rating, COUNT(rating) AS amount FROM film GROUP BY rating;

	rating	amount
▶	PG	194
	G	178
	NC-17	210
	PG-13	223
	R	195

```
SELECT rating, COUNT(rating) AS amount FROM film GROUP BY rating
ORDER BY amount;
```

	rating	amount
▶	G	178
	PG	194
	R	195
	NC-17	210
	PG-13	223

- 영화 등급별로 평균 대여 기간이 얼마인지 확인

```
SELECT rating, AVG(rental_duration) AS duration FROM film GROUP BY
rating;
```

	rating	duration
▶	PG	5.0825
	G	4.8371
	NC-17	5.1429
	PG-13	5.0538
	R	4.7744

- 분실 시 교체 비용이 가장 큰 영화와 가장 적은 영화 확인

```
SELECT MAX(replacement_cost), MIN(replacement_cost) FROM film;
```

- 각 영화의 제목도 함께 확인하고 싶다면 아래와 같이 쿼리 실행

```
SELECT title, MAX(replacement_cost) FROM film;
SELECT title, MIN(replacement_cost) FROM film;
```

- 최대, 최소값을 가진 행의 다른 열 값도 가져오고 싶을 때는 서브쿼리를 사용

```
SELECT title, replacement_cost FROM film
WHERE replacement_cost=(SELECT MAX(replacement_cost) FROM film)
# OR
replacement_cost=(SELECT MIN(replacement_cost) FROM film);
```

- **SELECT - PIVOT TABLE 만들기**

- SUM() 함수, CASE문, GROUP BY 활용

```
SELECT customer_id,  
SUM(CASE WHEN film.rating='G' THEN 1 END) AS 'G',  
SUM(CASE WHEN film.rating='PG' THEN 1 END) AS 'PG',  
SUM(CASE WHEN film.rating='PG-13' THEN 1 END) AS 'PG-13',  
SUM(CASE WHEN film.rating='R' THEN 1 END) AS 'R',  
SUM(CASE WHEN film.rating='NC-17' THEN 1 END) AS 'NC-17'  
FROM film, rental, inventory  
WHERE rental.inventory_id=inventory.inventory_id AND  
inventory.film_id=film.film_id  
GROUP BY customer_id;
```

- 이 경우는 각 rating 조건에 맞는 경우 1로 취급하고 SUM으로 더하라는 의미인데, 숫자가 1이므로 SUM 대신 COUNT를 넣어도 같은 결과 확인 가능

- **SELECT - HAVING 절**

- 집계함수를 사용한 결과에 조건을 부여하고자 할 때 WHERE가 아닌 HAVING에 조건 기술

```
SELECT customer_id,  
SUM(CASE WHEN film.rating='G' THEN 1 END) AS 'G',  
SUM(CASE WHEN film.rating='PG' THEN 1 END) AS 'PG',  
SUM(CASE WHEN film.rating='PG-13' THEN 1 END) AS 'PG-13',  
SUM(CASE WHEN film.rating='R' THEN 1 END) AS 'R',  
SUM(CASE WHEN film.rating='NC-17' THEN 1 END) AS 'NC-17'  
FROM film, rental, inventory  
WHERE rental.inventory_id=inventory.inventory_id AND  
inventory.film_id=film.film_id  
GROUP BY customer_id  
HAVING SUM(CASE WHEN film.rating='G' THEN 1 END)>=10 OR SUM(CASE WHEN film.rating='PG' THEN 1 END)>=10;
```

- 전체관람가(G)나 보호자의 시청 지도가 필요한 관람물(PG)을 10회 이상 대여한 고객 ID 확인 가능
 - 조건은 한 개만 줘도 됨(OR이후 부분 삭제하면 전체관람가 10회 이상 대여한 고객 ID만 반환)

• SELECT 형식 정리

```
SELECT select_expr
  [FROM table_references]
  [WHERE where_condition]
  [GROUP BY {col_name | expr | position}]
  [HAVING where_condition]
  [ORDER BY {col_name | expr | position}]
```

- JOIN
 - 조인(JOIN) - 2개 이상의 테이블을 묶어서 하나의 결과 테이블을 만드는 것
 - 일대다(one-to-many) 관계 - 예) 기업의 직원 테이블과 급여 테이블, 학교의 학생 테이블과 학점 테이블
 - 다대다(many-to-many) 관계 - 예) 한 학생은 여러 개의 동아리에 가입해서 활동할 수 있고, 하나의 동아리에는 여러 학생이 가입할 수 있음
- JOIN 대상 테이블
 - 고객 정보의 상세한 조회를 위해 customer, store, address, payment 테이블을 내부 조인과 외부 조인을 활용하기

```
SELECT * FROM customer;
SELECT * FROM store;
SELECT * FROM address;
```

- 조회해보면 customer 테이블에는 해당 고객이 주로 이용하는 지점이 지점의 '번호'로만 들어가 있고, 고객의 주소도 주소 '번호'로만 들어가 있는 것을 확인할 수 있음

- 내부 조인(inner join)
 - 조인 조건을 만족하는 행만을 포함하여 출력하는 조인

```
SELECT <열 목록>
FROM <첫 번째 테이블>
INNER JOIN <두 번째 테이블>
ON <조인될 조건>
[WHERE 검색조건];
```

```
SELECT customer_id, first_name, last_name, address, phone
FROM customer
INNER JOIN address
ON customer.address_id=address.address_id;
```

	customer_id	first_name	last_name	address	phone
1	MARY	SMITH	1913 Hanol Way		28303384290
2	PATRICIA	JOHNSON	1121 Loja Avenue		838635286649
3	LINDA	WILLIAMS	692 Joliet Street		448477190408
4	BARBARA	JONES	1566 Inegi Manor		705814003527
5	ELIZABETH	BROWN	53 IdRu Parkway		10659649674
6	JENNIFER	DAVIS	1795 Santiago de Compostela Way		860152626434
7	MARIA	MILLER	900 Santiago de Compostela Parkway		716571220373
8	SUSAN	WILSON	478 Joliet Way		657282385970
9	MARGARET	MOORE	613 Korolev Drive		380657322649
10	DOROTHY	TAYLOR	1531 Sal Drive		648896936185
11	LISA	ANDERSON	1542 Tarlac Parkway		635207277345
12	NANCY	THOMAS	808 Shopal Manor		465887807014
13	KAREN	JACKSON	270 Amroha Parkway		695479687538
14	BETTY	WHITE	770 Bydgoszcz Avenue		517338314235
15	HELEN	HARRIS	410 Ilkan Lane		990911107354
16	SANDRA	MARTIN	360 Toulouse Parkway		949312333307
17	DONNA	THOMPSON	270 Toulon Boulevard		407752414682
18	CAROL	GARCIA	320 Brest Avenue		747791394069
19	RUTH	MARTINEZ	1417 Lancaster Avenue		272572357893
20	SHARON	ROBINSON	1688 Okara Way		144453860132
21	MICHELLE	CLARK	262 A Conus (La Conus) Parkway		892775750053

```
SELECT customer_id, first_name, last_name, address, phone
FROM customer, address
WHERE customer.address_id=address.address_id; # 키값이 같은지 비교하는
형태의 '동등조인'
```

- 외부 조인(outer join)
 - 조인 조건을 만족하지 않는 행까지 포함하여 출력하는 조인
 - 외부 조인의 형식

```

SELECT <열 목록>
FROM <첫 번째 테이블(LEFT 테이블)>
    <LEFT | RIGHT> OUTER JOIN <두 번째 테이블(RIGHT 테이블)>
    ON <조인될 조건>
[WHERE 검색조건];

```

- 구매 이력이 없는 회원들의 정보도 함께 보고 싶을 때 외부 조인을 사용할 수 있음
 - 외부 조인이 아닌 내부 조인 시, 구매 기록이 있는 고객에 대한 정보만 확인할 수 있음

```

SELECT payment_id, payment.customer_id, first_name, last_name, rental_id, amount, payment_date FROM payment
INNER JOIN customer ON payment.customer_id=customer.customer_id;

```

```

SELECT payment_id, payment.customer_id, first_name, last_name, rental_id, amount, payment_date
FROM payment, customer
WHERE payment.customer_id=customer.customer_id;

```

• 참고 - INSERT문

- 테이블에 데이터를 삽입하는 명령어
- 작성 형식

```

INSERT [INTO] 테이블이름[(열1, 열2, ...)] VALUES (값1, 값2, ...)

```

- JOIN – 외부 조인 준비
 - 구매 이력이 없는 신규 회원 정보를 추가

```

INSERT INTO customer VALUES (600, 1, 'SUJIN','YOO','sujin.yoo@sakila.customer.org',1,1,now(),now());
INSERT INTO customer VALUES (601, 2, 'SUJIN','KIM','sujin.kim@sakila.customer.org',2,1,now(),now());

```

```
INSERT INTO customer VALUES (602, 1, 'SUJIN','PARK','sujin.park@sakilacustomer.org',3,1,now(),now());
INSERT INTO customer VALUES (603, 1, 'SUJIN','CHOI','sujin.choi@sakilacustomer.org',4,1,now(),now());
```

- 위 쿼리의 경우 테이블명 우측에 컬럼명을 기재하지 않음. 이런 경우에는 테이블 스키마의 실제 컬럼 순서와 동일하게 데이터를 작성해야 함
- 아래 쿼리로 잘 추가되었는지 확인

```
SELECT * FROM customer WHERE customer_id>599;
```

- 회원들 중, 구매 이력이 없는 회원 정보를 확인하고자 할 때 다음과 같은 쿼리를 통해 외부 조인 가능

```
SELECT payment_id, payment.customer_id, first_name, last_name, rental_id, amount, payment_date
FROM payment
RIGHT OUTER JOIN customer ON payment.customer_id=customer.customer_id;
```

```
SELECT payment_id, payment.customer_id, first_name, last_name, rental_id, amount, payment_date
FROM payment
RIGHT OUTER JOIN customer ON payment.customer_id=customer.customer_id
WHERE payment_id IS NULL;
```

- 여기서 payment 테이블이 LEFT, customer 테이블이 RIGHT에 해당. 조인 조건으로 사용하는 key값이 NULL이더라도 오른쪽 테이블(customer)의 값들을 보여달라는 의미의 쿼리

- **자체 조인(self join)**

- 자기 자신과 조인하는 것
- 자체 조인을 활용하는 대표적인 예는 조직도와 관련된 테이블



직원 이름(emp): 기본키	상관 이름(manager)	구내 번호(empTel)
나사장	없음(NULL)	0000
김재무	나사장	2222
김부장	김재무	2222-1
이부장	김재무	2222-2
우대리	이부장	2222-2-1
지사원	이부장	2222-2-2
이영업	나사장	1111
한과장	이영업	1111-1
최정보	나사장	3333
윤차장	최정보	3333-1
이주임	윤차장	3333-1-1

[실습] 자체 조인하기

- MySQL Workbench에서 아래 쿼리로 myDB 생성

```

DROP DATABASE IF EXISTS myDB;
CREATE DATABASE myDB;

```

• CREATE TABLE 문

- 새로운 테이블을 생성하고자 할 때 사용하는 명령어

```

CREATE TABLE 테이블명
(컬럼명1 자료형(자료크기) [NOT NULL] [PRIMARY KEY],
 컬럼명2 자료형(자료크기) [NOT NULL] [PRIMARY KEY],

```

```
...,  
컬럼명n 자료형(자료크기) [NOT NULL] [PRIMARY KEY])
```

- 기존에 존재하는 테이블의 데이터를 그대로 복사해서 새로운 테이블을 만들고 싶은 경우, 아래와 같은 형식으로 테이블 생성 가능

```
CREATE TABLE 새로운테이블 (SELECT 복사할열 FROM 기존테이블)
```

- 테이블 만들기
 - 조직도테이블 만들기

```
USE myDB;  
CREATE TABLE empTBL(emp CHAR(3), manager CHAR(3), empTel VAR  
CHAR(8));
```

- 조직도 테이블에 데이터 입력

```
INSERT INTO empTBL VALUES ('나사장', NULL, '0000');  
INSERT INTO empTBL VALUES ('김재무', '나사장', '2222');  
INSERT INTO empTBL VALUES ('김부장', '김재무', '2222-1');  
INSERT INTO empTBL VALUES ('이부장', '김재무', '2222-2');  
INSERT INTO empTBL VALUES ('우대리', '이부장', '2222-2-1');  
INSERT INTO empTBL VALUES ('지사원', '이부장', '2222-2-2');  
INSERT INTO empTBL VALUES ('이영업', '나사장', '1111');  
INSERT INTO empTBL VALUES ('한과장', '이영업', '1111-1');  
INSERT INTO empTBL VALUES ('최정보', '나사장', '3333');  
INSERT INTO empTBL VALUES ('윤차장', '최정보', '3333-1');  
INSERT INTO empTBL VALUES ('이주임', '윤차장', '3333-1-1');
```

- 자체 조인하기
 - 우대리 상관의 구대 번호 확인

```
SELECT A.emp AS '부하직원', B.emp AS '직속상관', B.empTel AS '직속상관  
연락처'  
FROM empTBL A  
INNER JOIN empTBL B
```

```
ON A.manager = B.emp
WHERE A.emp = '우대리';
```

	부하직원	직속상관	직속상관연락처
▶	우대리	이부장	2222-2

- JOIN

- UNION - 두 쿼리의 결과를 행으로 합치는 연산자
- 사용 형식

```
SELECT 문장1
UNION [ALL]
SELECT 문장2
```

- sakila 데이터베이스의 actor 테이블, customer 테이블의 모든 이름(first_name, last_name) 출력하기

```
USE sakila;

SELECT first_name, last_name
FROM actor
UNION
SELECT first_name, last_name
FROM customer;
```

- UPDATE

- 테이블에 입력되어 있는 값을 수정하는 명령어

```
UPDATE 테이블이름
SET 열1=값1, 열2=값2, ...
WHERE 조건;
```

- WHERE 절을 생략하면 테이블 전체의 값이 수정됨
- sakila 데이터베이스의 customer 테이블의 customer_id 603인 회원의 first_name을 수정하기

```
USE sakila;
UPDATE customer
SET first_name = 'MINA'
WHERE customer_id = 603;

SELECT * FROM customer WHERE customer_id=603;
```

• DELETE

- 테이블에 입력되어 있는 값을 삭제하는 명령어

```
DELETE FROM 테이블이름 WHERE 조건;
```

- WHERE 절을 생략하면 테이블 전체의 값이 삭제됨
 - 테이블 스키마는 남아있고 데이터만 사라짐
- sakila 데이터베이스의 customer 테이블의 customer_id가 601인 회원 정보 삭제

```
DELETE FROM customer WHERE customer_id=601;

SELECT * FROM customer WHERE customer_id=601;
```

파이썬과 MySQL 연동 실습

- 파이썬 라이브러리 pymysql 설치
- 한글 폰트를 위한 Matplotlib 설치
- 불러온 데이터를 dataframe으로 보여주기 위한 pandas 설치

```
pip install pymysql
pip install matplotlib
pip install pandas
```

<https://coduking.com/entry/파이썬-MySQL-연동/>

<https://hongong.hanbit.co.kr/파이썬과-mysql-데이터베이스-연동하기-pymysql-라이브러리-설/>

```
import pymysql
import matplotlib as plt
plt.rcParams['font.family'] = 'Malgun Gothic'
conn = pymysql.connect(host='localhost', user='root', password='비밀번호',
db='DB이름', charset='utf8')
cursor = conn.cursor()

sql = """select * from 테이블명 limit 5"""
cursor.execute(sql)
data = cursor.fetchall()
print(data)
conn.close()
```

```
import pymysql
import matplotlib as plt
plt.rcParams['font.family'] = 'Malgun Gothic'
conn = pymysql.connect(host='localhost', user='root', password='비밀번호',
db='myDB', charset='utf8')
cursor = conn.cursor()

cursor.execute("CREATE TABLE userTable (id char(4), userName char(15),
email char(20), birthYear int)")
cursor.execute("INSERT INTO userTable VALUES('hong' , '홍지윤' , 'hong@naver.com' , 1996)")
cursor.execute("INSERT INTO userTable VALUES('kim' , '김태연' , 'kim@dau m.net', 2011)")
cursor.execute("INSERT INTO userTable VALUES('star' , '별사랑' , 'star@par
```

```
an.com' , 1990)")
cursor.execute("INSERT INTO userTable VALUES('yang' , '양지은' , 'yang@g
mail.com' , 1993)")

conn.commit()
```

```
sql = """select * from userTable limit 5"""
cursor.execute(sql)
data = cursor.fetchall()
conn.close()
print(data)

import pandas as pd
df=pd.DataFrame(data, columns=['id', 'name', 'email', 'birth_year'])
print(df)
```

참고문헌

- 박우창,남송휘,이현룡. (2019). MySQL로 배우는 데이터베이스 개론과 실습. 한빛아카데미.
- 우재남. (2019). 데이터베이스 for Beginner. 한빛아카데미.